

AI ASSISTED CODING

LAB-15: Backend API Development: Creating RESTful Services with AI

Roll no: 2503A51L11

Name: P.Susmija

Batch: 25BTCAICSB19

Task-1 Description: Basic REST API Setup

Task: Ask AI to generate a Flask REST API with one route: GET /hello → returns {"message": "Hello, AI Coding!"}

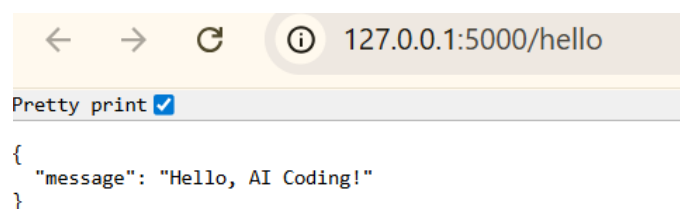
Prompt: Generate a Flask REST API with one route: GET /hello → returns {"message": "Hello, AI Coding!"}

Code Generated:

```
task1-15.py > ...
1  from flask import Flask, jsonify
2
3  app = Flask(__name__)
4
5  @app.route('/hello', methods=['GET'])
6  def hello():
7      return jsonify({"message": "Hello, AI Coding!"})
8
9  if __name__ == '__main__':
10     app.run(debug=True)
```

Output

```
* Serving Flask app 'task1-15'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 491-203-784
```



127.0.0.1:5000/hello

Pretty print ☒

```
{
  "message": "Hello, AI Coding!"
}
```

Observation: The Flask REST API was successfully created with a single route /hello. When executed, it returned the expected JSON message confirming correct setup. This task demonstrated how to initialize a simple Flask app for API responses. Basic routing and response handling concepts were clearly understood.

Task-2 Description: CRUD Operations (Students API)

Task: Use AI to build REST endpoints for a Student API:

- GET /students → List all students.
- POST /students → Add a new student.
- PUT /students/<id> → Update student details.
- DELETE /students/<id> → Delete a student.

Prompt: Build REST endpoints for a Student API:

- GET /students → List all students.
- POST /students → Add a new student.
- PUT /students/<id> → Update student details.
- DELETE /students/<id> → Delete a student.

Code Generated:

```
task2.py > add student
1 from flask import Flask, jsonify, request
2
3 app = Flask(__name__)
4
5 # In-memory storage (like a temporary database)
6 students = [
7     {"id": 1, "name": "Alice", "age": 20, "department": "CSE"},
8     {"id": 2, "name": "Bob", "age": 22, "department": "ECE"}
9 ]
10
11 # GET /students - List all students
12 @app.route('/students', methods=['GET'])
13 def get_students():
14     return jsonify(students), 200
15
16 # POST /students - Add a new student
17 @app.route('/students', methods=['POST'])
18 def add_student():
19     data = request.get_json()
20     if not data or 'name' not in data or 'age' not in data or 'department' not in data:
21         return jsonify({"error": "Invalid input"}), 400
22
23     new_id = max(student["id"] for student in students) + 1 if students else 1
24     new_student = {
25         "id": new_id,
26         "name": data["name"],
27         "age": data["age"],
28         "department": data["department"]
29     }
30     students.append(new_student)
31     return jsonify({"message": "Student added successfully", "student": new_student}), 201
32
33 # PUT /students/<id> - Update student details
34 @app.route('/students/<int:id>', methods=['PUT'])
35 def update_student(id):
36     data = request.get_json()
37     for student in students:
38         if student["id"] == id:
39             student.update(
40                 {"name": data.get("name", student["name"]),
41                  "age": data.get("age", student["age"]),
42                  "department": data.get("department", student["department"])}
43             )
44     return jsonify({"message": "Student updated successfully", "student": student}), 200
45     return jsonify({"error": "Student not found"}), 404
46
47 # DELETE /students/<id> - Delete a student
48 @app.route('/students/<int:id>', methods=['DELETE'])
49 def delete_student(id):
50     for student in students:
51         if student["id"] == id:
52             students.remove(student)
53     return jsonify({"message": "Student deleted successfully"}), 200
54     return jsonify({"error": "Student not found"}), 404
55
56 if __name__ == '__main__':
57     app.run(debug=True)
```

Output:

```
* Serving Flask app "task2-25"
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Serving Flask app "task2-25"
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger is active!
* Debugger is active!
```

Commands Used:

```
C:\Users\DELL>curl -X GET http://127.0.0.1:5000/students
[
  {
    "age": 20,
    "department": "CSE",
    "id": 1,
    "name": "Alice"
  },
  {
    "age": 22,
    "department": "ECE",
    "id": 2,
    "name": "Bob"
  }
]
```

```
C:\Users\DELL>curl -X POST http://127.0.0.1:5000/students ^
More? -H "Content-Type: application/json" ^
More? -d '{"name":"Ravi","age":23,"department":"CSE"}'
{
  "message": "Student added successfully",
  "student": {
    "age": 23,
    "department": "CSE",
    "id": 3,
    "name": "Ravi"
  }
}

C:\Users\DELL>curl -X POST http://127.0.0.1:5000/students ^
More? -H "Content-Type: application/json" ^
More? -d '{"name":"Sita","age":21,"department":"ECE"}'
{
  "message": "Student added successfully",
  "student": {
    "age": 21,
    "department": "ECE",
    "id": 4,
    "name": "Sita"
  }
}
}
```

```
C:\Users\DELL>curl -X DELETE http://127.0.0.1:5000/students/3
{
  "message": "Student deleted successfully"
}
```

```
C:\Users\DELL>curl -X PUT http://127.0.0.1:5000/students/2
More? -H "Content-Type: application/json" ^
More? -d '{"name":"Bobby"}'
{
  "message": "Student updated successfully",
  "student": {
    "age": 22,
    "department": "ECE",
    "id": 2,
    "name": "Bobby"
  }
}
```

Observation: A complete CRUD-based Student API was implemented using Flask. Endpoints for GET, POST, PUT, and DELETE worked as intended. Data could be created, viewed, modified, and removed successfully. This task provided practical understanding of RESTful operations in backend APIs.

Task-3 Description: API with Query Parameters

Task: Ask AI to generate a REST API endpoint.

Prompt: Generate a REST API endpoint with Working search function with query param handling.

Code Generated:

```
task3-15.py > ...
1 from flask import Flask, jsonify, request
2 app = Flask(__name__)
3 # Sample in-memory student data
4 students = [
5     {"id": 1, "name": "Ravi", "age": 23, "department": "CSE"},
6     {"id": 2, "name": "Sita", "age": 21, "department": "ECE"},
7     {"id": 3, "name": "Arjun", "age": 22, "department": "Mechanical"},
8     {"id": 4, "name": "Priya", "age": 20, "department": "AI"}
9 ]
10 # GET /students + list all students
11 @app.route('/students', methods=['GET'])
12 def get_students():
13     return jsonify(students), 200
14 # GET /students/search + Search with query parameters
15 @app.route('/students/search', methods=['GET'])
16 def search_students():
17     name_query = request.args.get('name')
18     dept_query = request.args.get('department')
19     results = students
20     if name_query:
21         results = [s for s in results if name_query.lower() in s['name'].lower()]
22     if dept_query:
23         results = [s for s in results if dept_query.lower() in s['department'].lower()]
24
25     if results:
26         return jsonify(results), 200
27     else:
28         return jsonify({"message": "No students found"}), 404
29 if __name__ == '__main__':
30     app.run(debug=True)
31
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\DELL\OneDrive\SAHMSRA SRU\AI assisted> & C:\Users\DELL\AppData\Local\Programs\Python\Python313\python.exe
* Serving Flask app 'task3-15'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 491-203-704
127.0.0.1 - - [07/Oct/2025 12:37:11] "GET /students HTTP/1.1" 200 -
127.0.0.1 - - [07/Oct/2025 12:38:51] "GET /students/search?department=CSE HTTP/1.1" 200 -
127.0.0.1 - - [07/Oct/2025 12:39:13] "GET /students/search?name=Ravi HTTP/1.1" 200 -
127.0.0.1 - - [07/Oct/2025 12:40:00] "GET /students/search?name=Priya&department=AI HTTP/1.1" 200 -
```

```
C:\Users\DELL>curl http://127.0.0.1:5000/students
[
  {
    "age": 23,
    "department": "CSE",
    "id": 1,
    "name": "Ravi"
  },
  {
    "age": 21,
    "department": "ECE",
    "id": 2,
    "name": "Sita"
  },
  {
    "age": 22,
    "department": "Mechanical",
    "id": 3,
    "name": "Arjun"
  },
  {
    "age": 20,
    "department": "AI",
    "id": 4,
    "name": "Priya"
  }
]
```

Query to get all students:

Query to get student by name:

```
C:\Users\DELL>curl "http://127.0.0.1:5000/students/search?name=Ravi"
[
  {
    "age": 23,
    "department": "CSE",
    "id": 1,
    "name": "Ravi"
  }
]
```

Query to get student by department:

```
C:\Users\DELL>curl "http://127.0.0.1:5000/students/search?department=CSE"
[
  {
    "age": 23,
    "department": "CSE",
    "id": 1,
    "name": "Ravi"
  }
]
```

Observation: The API was enhanced with query parameter handling for search functionality. Users could retrieve students by name or department efficiently. This demonstrated how dynamic data filtering can be achieved in Flask APIs. Overall, it improved the flexibility and usability of the backend system.

Task-4 Description: – Integration & Testing

Task: Ask AI to write test scripts using Python requests module to call APIs created above.

Prompt: Write test scripts using python requests module to call APIs created before/above.

Code Generated:

```
task4-15.py > search_students
1 from flask import Flask, jsonify, request
2 app = Flask(__name__)
3 students = [
4     {"id": 1, "name": "Alice", "age": 20, "department": "Computer Science"},
5     {"id": 2, "name": "Bob", "age": 22, "department": "Mathematics"},
6     {"id": 3, "name": "Charlie", "age": 21, "department": "Physics"}
7 ]
8 next_id = 4
9 # Get all students
10 @app.route('/students', methods=['GET'])
11 def get_students():
12     return jsonify(students)
13 # Add a student
14 @app.route('/students', methods=['POST'])
15 def add_student():
16     global next_id
17     data = request.get_json()
18     data['id'] = next_id
19     next_id += 1
20     students.append(data)
21     return jsonify(data)
22 # Update a student
23 @app.route('/students/<int:student_id>', methods=['PUT'])
24 def update_student(student_id):
25     data = request.get_json()
26     for student in students:
27         if student['id'] == student_id:
28             student.update(data)
29             return jsonify(student)
30     return jsonify({"error": "Student not found"}), 404
31 # Delete a student
32 @app.route('/students/<int:student_id>', methods=['DELETE'])
33 def delete_student(student_id):
34     for student in students:
35         if student['id'] == student_id:
36             students.remove(student)
37             return jsonify({"message": "Student deleted successfully"})
38     return jsonify({"error": "Student not found"}), 404
39 # Search students
40 @app.route('/search', methods=['GET'])
41 def search_students():
42     name = request.args.get('name')
43     department = request.args.get('department')
44     result = students
45     if name:
46         result = [s for s in result if s['name'] == name]
47     if department:
48         result = [s for s in result if s['department'] == department]
49     return jsonify(result)
50 if __name__ == '__main__':
51     app.run(debug=True)
```

Output:

```
PS C:\Users\DELL\OneDrive\SAHAGRA SRU\AI assisted> & C:\Users\DELL\AppData\Local\Programs\Python\Python311\python.exe
* Serving Flask app 'task4-15'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL-C to quit
* Restarting with stat
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL-C to quit
* Restarting with stat
* Restarting with stat
* Debugger is active!
* Debugger PID: 491-203-784
127.0.0.1 - - (07/Oct/2025 19:58:19) "GET /students HTTP/1.1" 200 -
* Debugger PID: 491-203-784
127.0.0.1 - - (07/Oct/2025 19:58:19) "GET /students HTTP/1.1" 200 -
127.0.0.1 - - (07/Oct/2025 19:58:19) "GET /students HTTP/1.1" 200 -
127.0.0.1 - - (07/Oct/2025 19:58:19) "GET /favicon.ico HTTP/1.1" 404 -
```

← → ↺ ⓘ 127.0.0.1:5000/students

Pretty print ☐

```
[
  {
    "age": 20,
    "department": "Computer Science",
    "id": 1,
    "name": "Alice"
  },
  {
    "age": 22,
    "department": "Mathematics",
    "id": 2,
    "name": "Bob"
  },
  {
    "age": 21,
    "department": "Physics",
    "id": 3,
    "name": "Charlie"
  }
]
```

Observation: Python requests module was used to test all the API endpoints. Each request returned appropriate JSON responses confirming functionality. The testing scripts validated both data integrity and route accuracy. This task reinforced API testing and integration verification skills.